Ontology COKB for Knowledge Representation and Reasoning in Designing Knowledge-based Systems

Nhon V. Do

University of Information Technology,
National University HoChiMinh City, Vietnam
nhondv@uit.edu.vn

Abstract. Knowledge Representation and Reasoning is at the heart of the great challenge of Artificial Intelligence, especially knowledge-based systems, expert systems and intelligent problem solvers. The models for knowledge such as logics, semantic networks, conceptual graphs, frames and classes are useful tools to design the above systems. However, they are not suitable to represent knowledge in the domains of reality applications. Designing of the knowledge bases and the inference engines of those systems in practice requires representations in the form of ontologies. For applications such as the intelligent problem solver in plane geometry, the knowledge contains a complicated system of concepts, relations, operators, functions, and rules. This situation motivates an ontology-based solution with such components. In this article, an ontology called Ontology of Computational Object Knowledge Base (COKB), will be presented in details. We also present a model for representing problems together reasoning algorithms for solving them, and design methods to construct applications. The above methodology has been used in designing many applications such as systems for solving analytic geometry problems, solving problems in plane geometry, and the expert system for diabetic micro vascular complication diagnosis.

Keywords: Knowledge representation, ontology, knowledge-based system, intelligent problem solver.

1 Introduction

Knowledge-Based Systems (KBSs) use knowledge-based techniques to support human decision-making, learning and action, solving problems. Knowledge representation methods play an important role in designing knowledge bases and inference engines of those systems. As mentioned in [1], first, general methods of knowledge representation and reasoning have been explored such as logics, semantic networks, conceptual graphs, rules of inference, frames and classes; and they can be found in [2-6]. These methods are useful, but they are very difficult to represent knowledge in domains of reality applications, because the complicated of knowledge domain. Second, researchers have developed specialized methods of knowledge representation and reasoning to handle core domains, such as time, space, causation and action. Third, researchers have tackled important applications of knowledge

representation and reasoning, including query answering, planning, the Semantic Web and intelligent problem solvers (IPSs). These intelligent systems must have suitable knowledge base and they not only give human readable solutions but also present solutions as the way people usually write them.

For applications such as the intelligent problem solver in plane geometry, the knowledge contains a system of structured concepts, relations, operators, functions, and rules; besides, knowledge reasoning requires complex symbolic computations. The system can solve problems in general forms. Users only declare hypothesis and goal of problems base on a simple language but strong enough for specifying problems. The practical methods in [7-9] are difficult to use for designing this system. This situation motivates an ontology-based solution with such components.

Ontologies became popular in computer science. Gruber [10] defines an ontology as an "explicit specification of a conceptualization". Several authors have made small adaptations to this, and some of its definition can be found in [11-13] and [3]. The followings are common definitions of the terminology ontology:

- "The subject *ontology* is the study of *categories* of things that exist or may exist in some domain. The product of such a study, called *an ontology*, is a catalog of the types of things that are assumed to exist in a domain of interest D from the perspective of a person who uses a language L for the purpose of talking about D" (John F. Sowa, [3])
- For Artificial Intelligence researchers "an ontology describes a formal, shared conceptualization of a particular domain of interest. Thus, ontologies provide a way of capturing a shared understanding of a domain that can be used both by humans and systems to aid in information exchange and integration" (L. Stojanovic, [11]).

The article [14] address the dependency of the applications on the societies by separately defining process's ontology, the agent's knowledge, the society's ontology, and the society's knowledge. The proposed system can be applied to cloud computing platform. A semantic web application was presented in [15], in which the authors did a research on semantic search. A model proposed for the exploitation of ontology-based KBs to improve search over large document repositories. The approach includes an ontology-based scheme for the semi-automatic annotation of documents, and a retrieval system. In [16], authors presented some principles for organizing a library of reusable ontological theories which can be configured into an application ontology. This application ontology is then exploited to organize the knowledge acquisition process and to support computational design. [17] proposed an extended model of the simple generic knowledge models, and used it for software experience knowledge. In [18], a knowledge-based system for the author linkage problem, called SudocAD, was constructed based on a method combining numerical and knowledge-based techniques. The ontology used in this system contains a hierarchy of concepts and a hierarchy of relations. [19] presented an ontology-based method of supplementing and verifying information stored in a framework system of the semantic knowledge base. Development of applications in [15-19] was based on commonly used ontologies with simple concepts and relationships among them. In [20, 21] the author presented advanced topics of ontological engineering and showed us a couple of concrete examples. It provides us with conceptual tools for analyzing

problems in a right way by which we mean analysis of underlying background of the problems as well as their essential properties to obtain more general and useful solutions.

Ontologies give us a modern approach for designing knowledge components of KBS. Practical applications such as the IPS in plane geometry expect an ontology consisting of knowledge components concepts, relations, relations, operators, functions, and rules that support symbolic computation and reasoning. In this article, we present an ontology called ontology of *Computational Object Knowledge Base* (COKB). It includes models, specification language, problems and deductive methods. The main approach for COKB is to integrate ontology engineering, object-oriented modeling ([22]) and symbolic computation programming ([23]). This ontology was used to produce applications in education and training such as a program for studying and solving problems in plane geometry presented in [24], a system that supports studying knowledge and solving of analytic geometry problems presented in [25], and the expert system for diabetic micro vascular complication diagnosis in [26]. It can be used to represent the total knowledge and to design the knowledge bases. Next, computational networks (Com-Net) and networks of computational objects (CO-Net) in [27] can be used for modeling problems and for designing inference engine.

The rest of the paper is organized as follows. In Section 2, ontology COKB will be explained consisting the model and specification language. Section 3 presents computational networks and networks of computational objects. Based on the models presented in previous sections, reasoning methods for solving problems will be discussed in Section 4. We will discuss how to design knowledge base and inference engine in Section 5, and section 6 gives some applications. Finally we conclude the paper in Section 7.

2 Ontology COKB

2.1 Model COKB

The model of computational objects knowledge bases, presented in [27], has been established from integration of ontology engineering, object-oriented modeling and symbolic computation programming.

Definition 1.1: A computational object (Com-object) O has the following characteristics:

(1) The set consists of attributes, denoted by Attrs(O).

(2) There are internal computational relations among attributes of Com-object O. These are manifested in the following features of the object:

- Given a subset A of Attrs(O). The object O can show us the attributes that can be determined from A.
- The object O will give the value of an attribute.
- It can also show the internal process of determining the attributes.

The structure of computational objects can be modeled by **(*Attrs, F, Facts, Rules*)**. *Attrs* is a set of attributes, *F* is a set of equations called computation relations, *Facts* is a set of properties or events of objects, and *Rules* is a set of deductive rules on facts.

Definition 1.2: The model of Computational Object Knowledge Base (COKB model) consists of six components:

(C, H, R, Ops, Funcs, Rules)

C is a set of concepts of computational objects, and each concept is a class of Com-objects. **H** is a set of hierarchy relation (IS-A relation) on the concepts. **R** is a set of another relations on C, and in case a relation r is a binary relation it may have properties such as reflexivity, symmetry, etc. The set **Ops** consists of *operators* on *C*. The set **Funcs** consists of functions on Com-Objects. The set **Rules** consists of deductive rules. The rules represent for statements, theorems, principles, formulas, and so forth. Rules can be written as the form "if {facts} then {facts}".

Facts must be classified so that the knowledge component *Rules* can be specified and processed in the inference engine of KBSs and IPSs. There are 12 kinds of facts have been proposed from the researching on real requirements and problems in different knowledge domains:

- **Fact of kind 1**: states information about object kind or type of an object.
- **Fact of kind 2**: states determination of an object or an attribute of an object.
- **Fact of kind 3**: states determination of an object or an attribute of an object by a value or a constant expression.
- **Fact of kind 4**: states equality on objects or attributes of objects.
- **Fact of kind 5**: states dependence of an object on other objects by an equation.
- **Fact of kind 6**: states a relation on objects or attributes of the objects.
- **Fact of kind 7**: states determination of a function.
- **Fact of kind 8**: states determination of a function by a value or a constant expression.
- **Fact of kind 9**: states equality between an object and a function.
- **Fact of kind 10**: states equality between a function and another function.
- **Fact of kind 11**: states dependence of a function on other functions or other objects by an equation.
- **Fact of kind 12**: states a relation on functions.

The basic technique for designing deductive algorithms is the unification of facts. Based on the kinds of facts and their structures, there will be criteria for unification proposed. Then it produces algorithms to check the unification of two facts.

2.2 Specification Language and Knowledge-based organization

The language for ontology COKB is constructed to specify knowledge bases with knowledge of the form COKB model. This language includes the following:

- A set of characters: letter, number, special letter.
- Vocabulary: keywords, names.
- Data types: basic types and structured types.
- Expressions and sentences.
- Statements.
- Syntax for specifying the components of COKB model.

The followings are some structures of definitions for facts, and functions.

Definitions of facts:

facts ::= **FACT**: fact-def+
fact-def ::= object-type | attribute | name | equation | relation | expression
object-type ::= cobject-type **(**name**)** | cobject-type **(**name <, name>* **)**
relation ::= relation **(** name <, name>+ **)**

Definitions of functions – form 1:

function-def ::= **FUNCTION** name;
ARGUMENT: argument-def+
RETURN: return-def;
[constraint]
[facts]
ENDFUNCTION;
return-def ::= name: type;

Definitions of functions – form 2:

function-def ::= **FUNCTION** name;
ARGUMENT: argument-def+
RETURN: return-def;
[constraint]
[variables]
[statements]
ENDFUNCTION;
statements ::= statement-def+
statement-def ::= assign-stmt | if-stmt | for-stmt
asign-stmt ::= name := expr;
if-stmt ::= **IF** logic-expr **THEN** statements+ **ENDIF**; |
IF logic-expr **THEN** statements+ **ELSE** statements+ **ENDIF**;
for-stmt ::= **FOR** name **IN [**range**] DO** statements+ **ENDFOR**;

From the model COKB and the specification language, the knowledge-based organization of the system will be established. It consists of structured text files that stores the components concepts, hierarchical relation, relations, operators, functions, rules, facts and objects. The knowledge base is stored by the files listed below.

- File "CONCEPTS.txt" stores names of concepts. For each concept, we have the corresponding file with the file name "<concept name>.txt", which contains the specification of this concept.
- File "HIERARCHY.txt" stores information of the component H of COKB model.
- Files "RELATIONS.txt" and "RELATIONS_DEF.txt" are structured text files that store the specification of relations.
- Files "OPERATORS.txt" and "OPERATORS_DEF.txt" are structured text files that store the specification of operators.
- Files "FUNCTIONS.txt" and "FUNCTIONS_DEF.txt" are structured text files that store the specification of functions.
- File "FACT_KINDS.txt" is the structured text file that stores the definition of

kinds of facts.

- File "RULES.txt" is the structured text file that stores deductive rules.
- Files "OBJECTS.txt" and "FACTS.txt" are the structured text files that store certain objects and facts.

3 Computational Network and Network of Computational Objects

Computational network (Com-Net) and network of computational objects (CO-Net) can be used to represent general problems in knowledge bases of the form COKB model. The methods and techniques for solving the problems on these networks will be useful tool for designing the inference engine of KBSs and IPSs.

3.1 Computational Network

Definition 3.1: A *computational network (CN)* is a structure (M, F) consisting of a set $M=\{x_1, x_2, ..., x_n\}$ variables and $F=\{f_1, f_2, ..., f_m\}$ is a set of computational relations over the variables in M. Each computational relation $f \in F$ has the following form:

(i) An equation over some variables in M, or

(ii) Deductive rule $f : u(f) \rightarrow v(f)$, with $u(f) \subseteq M$, $v(f) \subseteq M$, and there are corresponding formulas to determine (or to compute) variables in v(f) from variables in u(f). We also define the set $M(f) = u(f) \cup v(f)$.

The popular problem arising from reality applications is that to find a solution to determine a set $G \subseteq M$ from a set $H \subseteq M$. This problem is denoted by $H \rightarrow G$, H is the hypothesis and G is the goal of the problem.

Definition 3.2: Given a computational net K = (M, F). For $A \subseteq M$, $f \in F$, denote $f(A) = A \cup M(f)$ be the set obtained from A by applying f. Let $S=[f_1, f_2, ..., f_k]$ be a list of relations in F, the notation $S(A) = f_k(f_{k-1}(\dots f_2(f_1(A)) \dots))$ is used to denote the set of variables obtained from A by applying relations in S. We call S a *solution* of problem $H \rightarrow G$ if $S(H) \supseteq G$. Solution S is called a *good solution* if there is not a proper sublist S' of S such that S' is also a solution of the problem. The problem is *solvable* if there is a solution to solve it.

The following are some algorithms that show methods and techniques for solving the above problems on computational nets.

Algorithm 3.1: Find a solution of problem $H \rightarrow G$ on (M, F).

```
Step 1: Solution ← empty;   Solution_found ← false;
Step 2: Repeat
            Hold ← H;   Select  f ∈ F;
            while  not Solution_found  and (f found)  do
                if  (applying f from H produces new facts)
                    H ← H ∪ M(f);  Add f to Solution;
                if  (G ⊆ H)
                    Solution_found ← true;
                Select new  f ∈ F;
            Until   Solution_found  or  (H = Hold);
```

Algorithm 3.2: Find a good solution NewS from a solution S=[f_1, f_2, ..., f_k] of problem H→G.

Step 1: NewS ← []; V ← G;
Step 2: for i := k downto 1 do
If ($v(f_k) \cap V \neq \varnothing$)
Insert f_k at the beginning of NewS;
V ← (V – $v(f_k)$) ∪ ($u(f_k)$ – H);

3.2 Network of Computational Objects

Definition 3.3: A computational relation f between attributes or objects is called a *relation between the objects*. A network of Com-objects (CO-Net) consists of a set of Com-objects O = {O_1, O_2, ..., O_n} and a set of computational relations F = {f_1, f_2, ... , f_m}. This network of Com-objects is denoted by (O, F).

On network of Com-objects (O, F), we consider the problem that to determine attributes in set G from given attributes in set H. The problem is denoted by H→G.

Let M be a set of concerned attributes. Suppose A is a subset of M.

(a) For each f ∈ F, denote f(A) is the union of the set A and the set consists of all attributes in M deduced from A by f. Similarly, for each Com-object O_i ∈ O, O_i(A) is the union of the set A and the set consists of all attributes (in M) that the object O_i can determine from attributes in A.

(b) Suppose D = [t_1, t_2, ..., t_m] is a list of elements in F ∪ O. Denote A_0 = A, A_1 = $t_1(A_0)$, . . ., A_m = $t_m(A_{m-1})$, and D(A) = A_m.

We have $A_0 \subseteq A_1 \subseteq \ldots \subseteq A_m = D(A) \subseteq M$. Problem H→G is called *solvable* if there is a list D ⊆ F ∪ O such that D(H) ⊇ G. In this case, we say that D is a *solution* of the problem.

Algorithm 3.3: Find a solution of problem H→G on CO-Net (O,F).

Step 1: Solution ← empty list; Solution_found ← false;
Step 2: Repeat
Hold ← H; Select f ∈ F;
while not Solution_found and (f found) do
if (applying f from H produces new facts)
H ← H ∪ M(f); // M(f) is the set of attributes in relation f
Add f to Solution;
if (G ⊆ H)
Solution_found ← true;
Select new f ∈ F;
Until Solution_found or (H = Hold);
Step 3: if (not Solution_found)
Select O_i ∈ O such that O_i(H) ≠ H;
if (the selection is successful)
H ← O_i(H); Add O_i to Solution;
if (G ⊆ H)
Solution_found ← true;

else

goto step 2;

Example 3.1: In a triangle ABC, suppose that AB = AC, the values of the angle A and the edge BC are given (hypothesis). ABDE and ACFG are squares (outside triangle ABC). Compute EG.

The problem can be considered on the network of Com-objects (O, F) as follows:

$O = \{O_1$: triangle ABC with AB = AC, O_2 : triangle AEG, O_3 : square ABDE, O_4 : square ACFG $\}$, and $F = \{f_1, f_2, f_3, f_4, f_5\}$ consists of the following relations

f_1: $O_1.c = O_3.a$ {the edge c of triangle ABC = the edge of the square ABDE}

f_2: $O_1.b = O_4.a$ {the edge b of triangle ABC = the edge of the square ACFG}

f_3: $O_2.b = O_4.a$ {the edge b of triangle AEG = the edge of the square ACFG}

f_4: $O_2.c = O_3.a$ {the edge c of triangle AEG = the edge of the square ABDE}

f_5: $O_1.A + O_2.A = \pi$.

We have the problem $H \rightarrow G$, with $H = \{O_1.a, O_1.A\}$, and $G = \{O_2.a\}$;

$M(f_1) = \{ O_1.c , O_3.a \}$, $M(f_2) = \{ O_1.b , O_4.a \}$, $M(f_3) = \{ O_2.b , O_4.a \}$,

$M(f_4) = \{ O_2.c , O_3.a \}$, $M(f_5) = \{ O_1.\alpha , O_2.\alpha \}$,

$M = \{ O_1.a, O_1.b, O_1.c, O_1.A, O_2.b, O_2.c, O_2.A , O_2.a, O_3.a, O_4.a \}$.

The above algorithms will produce the solution $D = \{ f_5, O_1, f_1, f_2, f_3, f_4, O_2\}$, and the process of extending the set of attributes as follows:

$$A_0 \xrightarrow{f_5} A_1 \xrightarrow{O_1} A_2 \xrightarrow{f_1} A_3 \xrightarrow{f_2} A_4 \xrightarrow{f_3} A_5 \xrightarrow{f_4} A_6 \xrightarrow{O_2} A_7$$

with $A_0 = A = \{O_1.a , O_1.A\}$,

$A_1 = \{O_1.a , O_1.A, O_2.A\}$,

$A_2 = \{ O_1.a , O_1.A, O_2.A, O_1.b, O_1.c \}$,

$A_3 = \{O_1.a , O_1.A, O_2.A, O_1.b, O_1.c, O_3.a\}$,

$A_4 = \{O_1.a , O_1.A, O_2.A, O_1.b, O_1.c, O_3.a, O_4.a\}$,

$A_5 = \{O_1.a , O_1.A, O_2.A, O_1.b, O_1.c, O_3.a, O_4.a, O_2.b\}$,

$A_6 = \{O_1.a , O_1.A, O_2.A, O_1.b, O_1.c, O_3.a, O_4.a, O_2.b, O_2.c\}$,

$A_7 = \{O_1.a , O_1.A, O_2.A, O_1.b, O_1.c, O_3.a, O_4.a, O_2.b, O_2.c, O_2.a\}$.

3.3 Extensions

There are domains of knowledge based on a set of elements, in which each element can be a simple valued variables or a function. For example, in the knowledge of alternating current the alternating current intensity i(t) and the alternating potential u(t) are functions. It requires some extensions of networks of computational objects such as *extensive computational objects networks* defined below.

Definition 3.4: An *extensive computational Object* (ECom-Object) is an object O has structure including:

- A set of attributes $Attrs(O) = M_v \cup M_f$, with M_v is a set of simple valued variables; M_f is a set of functional variables. Between the variables (or attributes) there are internal relations, that are deduction rules or the computational relations.
- The object O has behaviors of reasoning and computing on attributes of objects or facts such as: find the closure of a set $A \subset Attrs(O)$; find a solution of problems which has the form $A \rightarrow B$, with $A \subseteq Attrs(O)$ and $B \subseteq Attrs(O)$; perform computations; consider determination of objects or facts.

Definition 3.5: An *extensive network of computational objects* is a model **(O, M, F, T)** that has the components below.

- $O = \{O_1, O_2, \ldots, O_n\}$ is the set of extensive computational objects.
- M is a set of object attributes. We will use the following notations: $M_v(O_i)$ is the set of simple valued attributes of the object O_i, $M_f(O_i)$ is the set of functional attributes of O_i, $M(O_i) = M_v(O_i) \cup M_f(O_i)$, $M(O) = M(O_1) \cup M(O_2) \cup \ldots \cup M(O_n)$, and $M \subseteq M(O)$.
- F is the set of computational relations on attributes in M and on objects in O.
- $T=\{t_1, t_2, \ldots, t_k\}$ is set of operators on objects.

On the structure (O,T), there are expressions of objects. Each expression of objects has its attributes as if it is an object.

4 Problem Solving on COKB

In design process for KBSs, after representing knowledge and organizing the knowledge base, the inference engine has to be designed. To design the inference engine of a KBS for solving problems in general, with the knowledge base modeled by ontology COKB, we need to represent problems in general form. Problems can be modeled by using networks of computational objects, and the hypothesis of a problem has the form (O, F), in which O is a set of Com-objects, F is a set of facts on objects.

4.1 Modeling Problems

Definition 4.1: Model of problems on COKB has the form (O, F) → G, and it consists of three sets: $O = \{O_1, O_2, \ldots, O_n\}$, $F = \{f_1, f_2, \ldots, f_m\}$, $G = \{g_1, g_2, \ldots, g_p\}$. The CO-Net (O,F) is the hypothesis of the problem: O consists of n computational objects, F is the set of facts given on the objects. G consists of goals of the problem, each goal may be any facts such as determine an object or an attribute (or some attributes) of an object, prove a relation between objects, compute a value of a function relative to objects.

Definition 4.2: Give a problem (O, F) → G on model COKB, and the goal G is to determine (or compute) attributes; M is the set of attributes considered in the problem, $A \subseteq M$. Denote L be the set of facts in the hypothesis.

- Each $f \in$ Rules, each $O_i \in O$, we define:
  $f(A) = A \cup M_A(f)$, with $M_A(f)$ is set of attributes can be deduced from A by f.
  $O_i(A) = A \cup M_A(O_i)$, with $M_A(O_i)$ is set of attributes deduced from A by O_i.
- Suppose $D = [r_1, r_2, \ldots, r_m]$ is a list of elements r_j, $r_j \in$ Rules or $r_j \in O$. Denote:
  $L_0 = L$, $L_1 = r_1(L_0)$, $L_2 = r_2(L_1)$, …. $L_m = r_m(L_{m-1})$ and $D(L) = L_m$

A problem is called *solvable* if there is a list D such that $G \subseteq D(L)$. In this case, we say that D is a *solution* of the problem.

4.2 Reasoning Methods

The basic technique for designing deductive algorithms is the unification of facts. Based on the kinds of facts and their structures, there will be criteria for unification proposed. Then it produces algorithms to check the unification of two facts.

The forward chaining strategy can be used with some techniques such as usage of heuristics, sample problems. The most difficult thing is modeling for experience, sensible reaction and intuitional human to find heuristic rules, so that reasoning algorithms are able to imitate the human thinking for solving problems. We can use network of computational objects, and its extensions to model problems; and use above techniques to design algorithms for automated reasoning. For instance, a reasoning algorithm for model COKB with sample problems can be briefly presented below.

Definition 4.3: Given a model COKB K = (C, H, R, Ops, Funcs, Rules), and a CO-Net (O, F) on this model. Denote L be the set of facts in the hypothesis. It is easy to verify that there exists an unique maximum set $\overline{L}$ such that the problem (O, F) → $\overline{L}$ is *solvable*. The set $\overline{L}$ is called the *closure* of L.

Lemma: Given a COKB model K = (C, H, R, Ops, Funcs, Rules), and a CO-Net (O, F) on this model, L is the set of facts in the hypothesis. By definition 4.2, problem (O, F) → $\overline{L}$ is solvable.

Then, there exists a list D = [r_1, r_2, ..., r_k] such that D(L) = $\overline{L}$.

Theorem 4.1: Given a COKB model K = (C, H, R, Ops, Funcs, Rules), and a problem S = (O, F) → G on this model. Denote L be the set of facts in the hypothesis. The following statements are equivalent.

(i) Problem S is solvable.

(ii) G ⊆ $\overline{L}$

(iii) There exists a list D such that G ⊆ D(L)

Proof:

* The equivalent of (i) and (iii) can been checked easily. It is definition about solvable.

* (i) => (ii): Problem S is solvable, but by definition 3.4, $\overline{L}$ is a maximum set such that problem (O, F) → $\overline{L}$ is solvable, so G ⊆ $\overline{L}$.

* (ii) => (iii): Problem (O, F) → $\overline{L}$ is solvable, by lemma, there exists an relation D = [f_1, f_2, ..., f_k] such that D(L) = $\overline{L}$. But G ⊆ $\overline{L}$, so G ⊆ D(L).

Theorem 4.1 shows that forward chaining reasoning will deduce to goals of problems. So the effectiveness of the following algorithms would be ensure.

Definition 4.4: Give knowledge domain K = (C, H, R, Ops, Funcs, Rules), Sample Problem (SP) is a model on knowledge K, it consists of two components as **(H_p, D_p)**. **H_p** is hypothesis of SP, and it has the following structure H_p = (O_p, F_p), with O_p is the set of Com-Objects of Sample Problem, F_p is the set of facts given on objects in O_p. **D_p** is a list of elements can be applied on F_p.

A model of Computational Object Knowledge Base with Sample Problems (COKB-SP) consists of 7 components: (C, H, R, Ops, Funcs, Rules, Sample); in which, (C, H,

R, Ops, Funcs, Rules) is knowledge domain which presented by COKB model, the Sample component is a set of Sample Problems of this knowledge domain.

Algorithm 4.1: To find a solution of problem P modelled by (O,F)→G on knowledge K of the form COKB-SP.

```
Solution ← empty list;   Solution_found ← false;
while (not Solution_found) do
    Find a Sample Problem can be applied to produce new facts;
    (heuristics can be used for this action)
    if (a sample S found)
        use the sample S to produce new facts;
        update the set of known facts, and add S to Solution;
        if (goal G obtained)
            Solution_found ← true;
        continue;
    Find a rule can be applied to produce new facts or new objects;
    (heuristics can be used for this action)
    if (a rule r found)
        use r to produce new facts or objects;
        update the set of known facts and objects, and add r to Solution;
        if (goal G obtained)
            Solution_found ← true;
end do; { while }
if (Solution_found)
    Solution is a solution of the problem;
else
    There is no solution found;
```

Algorithm 4.2: To find a sample problem.

Given a set of facts H on knowledge K of the form COKB-SP. Find a sample problem that can be applied on H to produce new facts.

```
SP ← Sample
Sample_found← false
Repeat
    Select S in SP, then consider S as a rule;
    (heuristics can be used for this action)
    if  (S can be applied on H to produce new facts)
        Sample_found  ← true;
        break;
    SP ← SP – {S};
Until SP = {};
if  (Sample_found)
    S is a sample problem can be applied on H to produce new facts;
else
    There is no sample problem found;
```

Heuristic rules help finding solution quickly and giving a good human-readable solution. The followings are some useful heuristic rules we used:

1. Priority use of rules for determining objects and attributes.
2. Transform objects to objects at a higher level in the hierarchical graph if there are enough facts. For example, a triangle will become isosceles triangle if it has two equal edges.
3. Use rules for producing new objects that contains elements, which are not in existing objects.
4. Use rules for producing new objects that have relationship with existing objects, especially the goal, in necessary situations.
5. Try to use deduction rules to get new facts, especially facts that have relationship with the goal.
6. If we could not produce new facts or new objects then we should use parameters and equations.
7. There are always new facts (relations or expressions) when we produce new objects.

4.3 Extension

In section 3, the extension of CO-Net (def. 3.4 and def. 3.5) is a model in which the concept of computational objects have functional attributes. Besides, objects have not only the inner computation relations but also the operators on objects.

Each expression of objects always has its attributes as if it is an object. Let E(Onet) is the set of expressions of objects on the network Onet = (O,M,F,T). We will use the following notations : M_E is set of attributes of expression E, E(r) is set of attributes in relation r of E(Onet), F_E is set of relations in E(ONet) or the relations attributes of the different objects. To solve problems modelled by using extensive CO-Net, we also define the concepts solutions and good solutions of problems. Then, the algorithm for solving problems is as follows:

Algorithm 4.3: Find a solution of the problem H→G.

Given Onet, E(ONet) and hypothesis $H \subseteq M \cup M_E$, $G \subseteq M \cup M_E$ that should be determined.

Determine E(ONet) and new relation of E(ONet).
Solution ← empty; Goal ← G ;
Repeat Hold ← H ;
  < ***Search Function choices a r ∈ F with Heuristic's law*** > ;
  while (**not** Solution_found **and** (r found))
    if (Having a new r)
      H ← H ∪ M(r); Solution ← Solution ∪ {r};
    if (G ⊆ H)
      Solution_found ← true;
    <***Search Function choice a r ∈ F with Heuristic's law*** >;
Until Solution_found **or** (H = Hold);
if (not Solution_found) <Not having a solution>;

e**lse** Solution is a solution of problem;

5 Designing knowledge base and inference engine

After collecting real knowledge, the knowledge base design includes the following tasks.

- The knowledge domain collected, which is denoted by K, will be represented or modeled based on the knowledge model COKB, CO-Net and their extensions or restrictions known. Some restrictions of COKB model were used to design knowledge bases are the model COKB lacking operator component (C, H, R, Funcs, Rules), the model COKB without function component (C, H, R, Ops, Rules), and the simple COKB sub-model (C, H, R, Rules). From Studying and analyzing the whole of knowledge in the domain, It is not difficult to determine known forms of knowledge presenting, together with relationships between them. In case that knowledge K has the known form such as COKB model, we can use this model for representing knowledge K directly. Otherwise, the knowledge K can be partitioned into knowledge sub-domains K_i (i = 1, 2, …, n) with lower complexity and certain relationships, and each K_i has the form known. The relationships between knowledge sub-domains must be clear so that we can integrate knowledge models of the knowledge sub-domains later.
- Each knowledge sub-domain K_i will be modeled by using the above knowledge models, so a knowledge model $M(K_i)$ of knowledge K_i will be established. The relationships on $\{K_i\}$ are also specified or represented. The models $\{M(K_i)\}$ together with their relationships are integrated to produce a model M(K) of the knowledge K. Then we obtain a knowledge model M(K) for the whole knowledge K of the application.
- Next, it is needed to construct a specification language for the knowledge base of the system. The COKB model and CO-Nets have their specification language used to specify knowledge bases of these form. A restriction or extension model of those models also have suitable specification language. Therefore, we can easily construct a specification language L(K) for the knowledge K. This language gives us to specify components of knowledge K in the organization of the knowledge base.

From the model M(K) and the specification language L(K), the knowledge base organization of the system will be established. It consists of structured text files that stores the knowledge components: concepts, hierarchical relation, relations, operators, functions, rules, facts and objects.

The inference engine design includes the following tasks:

- From the collection of problems with an initial classification, we can determine classes of problems base on known models such as CO-Net, and its extensions. This task helps us to model classes of problems as frame-based problem models, or as CO-Nets for general forms of problems. Techniques for modeling problems are presented in section 3. Problems modeled by using network of computational objects has the form (O, F)→Goal.

- The basic technique for designing deductive algorithms is the unification of facts. Based on the kinds of facts and their structures, there will be criteria for unification proposed. Then it produces algorithms to check the unification of two facts.
- To design reasoning algorithms for solving general problems, we can use the methods presented in sections 3 and 4. More works to do are to determine sample problems, heuristic rules (for selection of rules, for selection of sample problems, etc.).

6 Applications

The ontology COKB and design method for knowledge-based systems presented in previous sections have been used to produce many applications. In this section, we introduce some applications and examples about solutions of problems produced by the systems.

- The system that supports studying knowledge and solving analytic geometry problems. It consists of three components: the interface, the knowledge base, the knowledge processing modules or the inference engine. The program has menus for users searching knowledge they need and they can access knowledge base. Besides, there are windows for inputting problems. Users are supported a simple language for specifying problems. There are also windows in which the program shows solutions of problems and figures.
- The program for studying and solving problems in plane geometry. It can solve problems in general forms. Users only declare hypothesis and goal of problems base on a simple language but strong enough for specifying problems. The hypothesis can consist of objects, facts on objects or attributes, and parameters. The goal can be to compute an attribute, to determine an object or parameters, a relation or a formula. After specifying a problem, users can request the program to solve it automatically or to give instructions that help them to solve it themselves. The program also gives a human readable solution, which is easy to read and agree with the way of thinking and writing by students and teachers.

Examples below illustrate the functions of the above systems for solving problems in general forms.

Example 6.1: Given two points P(2, 5) and Q(5,1). Suppose d is a line that contains the point P, and the distance between Q and d is 3. Find the equation of line d.

Specification of the problem:

Objects = {[P, point], [Q, point], [d, line]}.
Hypothesis = {DISTANCE(Q, d) = 3, P = [2, 5], Q = [5, 1], ["BELONG", P, d]}.
Goal = [d.f].

Solution found by the system:

Step 1: {P = [2, 5]} $\Rightarrow$ {P}.
Step 2: {DISTANCE(Q, d) = 3} $\Rightarrow$ {DISTANCE(Q, d)}.
Step 3: {d, P} $\rightarrow$ {2d[1]+5d[2]+d[3] = 0}.

Step 4: {DISTANCE(Q, d) = 3} $\Rightarrow \frac{|5d[1]+d[2]+d[3]|}{\sqrt{d[1]^2+d[2]^2}} = 3$.

Step 5: {d[1] = 1, 2d[1] + 5d[2] + d[3] = 0, $\frac{|5d[1]+d[2]+d[3]|}{\sqrt{d[1]^2+d[2]^2}} = 3$ }

$\Rightarrow$ {d.f = ($x + \frac{24}{7}y - \frac{134}{7} = 0$), d.f = (x – 2 = 0)}.

Step 6: {d.f = $x + \frac{24}{7}y - \frac{134}{7} = 0$, d.f = x - 2= 0} $\Rightarrow$ {d.f}

Example 6.2: Given the parallelogram ABCD. Suppose M and N are two points of segment AC such that AM = CN. Prove that two triangles ABM and CDN are equal.

Specification of the problem:

Objects = {[A, POINT], [B, POINT], [C, POINT], [D, POINT], [M, POINT], [N, POINT], [O1, PARALLELOGRAM[A, B, C, D], [O2, TRIANGLE[A, B, M]], [O3, TRIANGLE [C, D, N]]}.

Hypothesis = { [« BELONG », M, SEGMENT[A, C]], [« BELONG », N, SEGMENT[A, C]], SEGMENT[A, M] = SEGMENT[C, N] }.

Goal = { O2 = O3}.

Solution found by the system:

Step 1: Hypothesis
$\Rightarrow$ { O2.SEGMENT[A, M] = O3.SEGMENT[C, N], O2.SEGMENT[A, B] = O1.SEGMENT[A, B], O3.SEGMENT[C, D] = O1.SEGMENT[C, D]}.

Step 2: Produce new objects related to O2, O3, and O1
$\Rightarrow$ {[O4, TRIANGLE[A, B, C]], [O5, TRIANGLE[C, D, A]]}.

Step 3: {[O1, PARALLELOGRAM[A, B, C, D]}
$\Rightarrow$ {O4 = O5, SEGMENT[A, B] = SEGMENT[C, D]}.

Step 4: { O2.SEGMENT[A, B] = O1.SEGMENT[A, B], O3.SEGMENT[C, D] = O1.SEGMENT[C, D], SEGMENT[A, B] = SEGMENT[C, D]}
$\Rightarrow$ {O2.SEGMENT[A, B] = O3.SEGMENT[C, D]}.

Step 5:{[«BELONG», M, SEGMENT[A, C]]} $\Rightarrow$ {O4.angle_A = O2.angle_A}.

Step 6: {[«BELONG», N, SEGMENT[A, C]]} $\Rightarrow$ {O5.angle_A = O3.angle_A }.

Step 7: {O4 = O5 } $\Rightarrow$ {O4.angle_A = O5.angle_A}.

Step 8: { O4.angle_A = O2.angle_A, O5.angle_A = O3.angle_A , O4.angle_A = O5.angle_A } $\Rightarrow$ { O2.angle_A = O3.angle_A}.

Step 9: { O2.SEGMENT[A, M] = O3. SEGMENT[C, N], O2.SEGMENT[A, B] = O3.SEGMENT[C, D], O2.angle_A = O3.angle_A } $\Rightarrow$ {O2 = O3}.

7 Conclusion and future work

The ontology COKB can be used to design and to implement knowledge-based

systems, especially expert systems and intelligent problem solvers. It consists of the model COKB, the specification language for COKB, the network of Com-Objects for modeling problems, algorithms for automated problem solving. It provides a natural way for representing knowledge domains in practice. By integration of ontology engineering, object-oriented modeling and symbolic computation programming it provides a highly intuitive representation for knowledge. These are the bases for designing the knowledge base of the system. The knowledge base is convenient for accessing and for using by the inference engine. The design of inference engine is easy and more effectively by using heuristics, sample problems in reasoning. So, the methods of modeling problems and algorithms for automated problem solving represent a normal way of human thinking. Therefore, systems not only give human readable solutions of problems but also present solutions as the way people write them.

Ontology COKB is a useful tool and method for designing practical knowledge bases, modeling complex problems and designing algorithms to solve automatically problems based on a knowledge base. It was used to produce intelligent softwares for e-learning. Besides applications were presented here, ontology COKB is able to use in other domain of knowledge such as physics and chemistry. Moreover, it also has been used to develop applications for e-government.

This ontology certainly has its limitation. Our future works are to develop the models and algorithms. First, Six related knowledge components in COKB model are complicated and not effective for representing simple knowledge domains. So, we need to investigate restriction models of COKB and their relationships and integration. Second, the concept Com-object should be extended so that attributes can be functional attributes. Third, the rules component of COKB model should have rules of the form equations to adapt more knowledge domains.

References

1. Frank van Harmelen & Vladimir Lifschitz & Bruce Porter, *Handbook of Knowledge Representation*, Elsevier, 2008.
2. Stuart Russell & Peter Norvig, *Artificial Intelligence – A modern approach* (third edition), Prentice Hall, 2010.
3. John F. Sowa. *Knowledge Representation: Logical, Philosophical and Computational Foundations*, Brooks/Cole, 2000.
4. George F. Luger, *Artificial Intelligence: Structures And Strategies For Complex Problem Solving*. Addison Wesley Longman, Inc. 2008.
5. Chitta Baral, *Knowledge Representation, Reasoning and Declarative Problem Solving*, Cambridge University Press, 2003.
6. Lakemeyer, G. & Nebel, B. *Foundations of Knowledge representation and Reasoning*, Berlin Heidelberg: Springer-Verlag, 1994.
7. Wen-tsun Wu, *Mechanical Theorem Proving in Geometries*, Springer-Verlag, 1994.
8. Chou, S.C. & Gao, X.S. & Zhang, J.Z. *Machine Proofs in Geometry*, Singapore: Utopia Press, 1994.
9. Pfalzgraf, J. & Wang, D. *Automated Practical Reasoning*, NewYork: Springer-Verlag, 1995.
10. Gruber, T. R., Toward Principles for the Design of Ontologies Used for Knowledge Sharing, *International Journal Human-Computer Studies* **43** (1995), 907-928.

11. L. Stojanovic, J. Schneider, A. Maedche, S. Libischer, R. Suder, T. Lumpp, A. Abecker, G. Breiter, J. Dinger, The Role of Ontologies in Autonomic Computing Systems, *IBM Systems Journal* **43** (2004), 598-616.
12. Asunción Gómez-Pérez & Mariano Férnandez-López & Oscar Corcho, *Ontological Engineering*, Springer-Verlag, 2004.
13. Guarino, N. Formal Ontology, Conceptual Analysis and Knowledge Representation, *International Journal of Human-Computer Studies*, **43** (1995), 625–640.
14. Mahmood Fathalipour, Ali Selamat, and Jason J. Jung, Ontology-based, process-oriented, and society-independent agent system for cloud computing, *International Journal of Distributed Sensor Networks*, Volumn 2014 (2014).
15. David Vallet, Miriam Fernández, and Pablo Castells, An Ontology-Based Information Retrieval Model, In book: *The Semantic Web, Research and Applications*. LNCS 3532 (2005), 455-470, Springer-Verlag Berlin Heidelberg, 2005.
16. G. VAN HEIJST, A. T H. S CHREIBER AND B. J. W IELINGA, Using explicit ontologies in KBS development. *Int. J. Human – Computer Studies* **45** (1997), 183 – 292, Academic Press Limited, 1997.
17. Abdulmajid H. Mohamed, Sai Peck Lee, Siti Salwah Salim, An Ontology-Based Knowledge Model for Software Experience Management. *International Journal of The Computer, the Internet and Management* Vol. 14. No.3 (2006), 79-88.
18. Michel Chein & Michel Leclere & Yann Nicolas, SudocAD : A Knowledge-Based System for the Author Linkage Problem, *Proceedings of the Fifth International Conference on Knowledge and Systems Engineering*, Vol. 1 (2014), 65-83, Springer International Publishing Switzerland, 2014.
19. Marek Krotkiewics & Krystian Wojtkiewicz, An introduction to Ontology Based Structured Knowledge Base System: Knowledge Acquisition Module, A. Selamat et al. (Eds.): *Intelligent Information Database Systems*, Part I, LNAI 7802 (2013), 497-506. Springer-Verlag Berlin Heidelberg, 2013.
20. Riichiro Mizoguchi, *Advanced course of Ontological Engineering*, New Generation Computing, Vol. 22, Issue 2 (2004), 193-220, Ohmsha Ltd. and Springer-Verlag, 2004.
21. Riichiro Mizoguchi, What Ontological Engineering can do for Solving Real-World Problems. *Proceedings of the Fifth International Conference on Knowledge and Systems Engineering*, Vol. 1 (2014), 3, Springer International Publishing Switzerland, 2014.
22. Berge, J.M. & Levia, O. & Rouillard, J. *Object-Oriented Modeling*, Netherlands: Kluwer Academic Publishers, 1996.
23. L. Bernadin et al., *Maple Programming Guide*, Maplesoft, a division of Waterloo Maple Inc. 2012.
24. Do Van Nhon, A Program for studying and Solving problems in Plane Geometry, *Proceedings of International Conference on Artificial Intelligence*, Las Vegas, USA (2000), 1441-1447.
25. Do Van Nhon, A system that supports studying knowledge and solving of analytic geometry problems, *16th World Computer Congress 2000, Proceedings of Conference on Education Uses of Information and Communication Technologies*, Beijing, China (2000), 236-239.
26. Truc Ly Nguyen Thi & Nhon Van Do, *An Expert System for Diabetic Microvascular Complication Diagnosis*. Journal of Computer Science Issues, Vol. 10, Issue 4, No 2, (2013), 308-316.
27. Nhon Van Do, Model for Knowledge Bases of Computational Objects, *International Journal of Computer Science Issues*, Vol. 7, Issue 3, No 8, (2010), 11-20.